of functionality. Each area can contain its own Pages folder, which is analogous to the main Pages folder in your application.

DEFINITION *Areas* are used to group Razor Pages into separate hierarchies for organizational purposes. I rarely use areas and prefer to create subfolders in the main Pages folder instead. The one exception is the Identity UI, which uses a separate Identity area by default. For more details on areas, see Microsoft's "Areas in ASP.NET Core" documentation: http://mng.bz/7Vw9.

The Microsoft.AspNetCore.Identity.UI package creates Razor Pages in the Identity area. You can override any page in this default UI by creating a corresponding page in the Areas/Identity/Pages folder in your application. In figure 23.6, the default template adds a _ViewStart.cshtml file that overrides the template that is included as part of the default UI. This file contains the following code, which sets the default Identity UI Razor Pages to use your project's default _Layout.cshtml file:

```
@{
    Layout = "/Pages/Shared/_Layout.cshtml";
}
```

Some obvious questions at this point are "How do you know what's included in the default UI?" and "Which files can you override?" You'll see the answers to both in section 23.5, but in general you should try to avoid overriding files where possible. After all, the goal with the default UI is to reduce the amount of code you have to write!

The Data folder in your new project template contains your application's EF Core `DbContext`, called `ApplicationDbContext`, and the migrations for configuring the database schema to use Identity. I'll discuss this schema in more detail in section 23.3.3.

The final additional file included in this template compared with the no-authentication version is the partial Razor view Pages/Shared/_LoginPartial.cshtml. This provides the Register and Login links